

Méthodes d'apprentissage

Dans la littérature, les méthodes d'apprentissage peuvent se structurer en trois catégories : i) l'apprentissage supervisé, ii) l'apprentissage par renforcement, et iii) l'apprentissage non supervisé. Les travaux pratiques proposés dans ce texte sont orientés vers l'**apprentissage supervisé**. Les techniques utilisées permettent l'identification automatique des hyperparamètres d'un réseau de neurones (ex. poids et biais) à partir des données en entrée et des sorties attendues. Le processus d'apprentissage vise donc à réduire l'erreur en sortie du réseau par rapport aux valeurs attendues.

Les problèmes abordés par l'apprentissage supervisé peuvent se concentrer sur divers objectifs. Ces objectifs peuvent être la régression, la classification, la labellisation (classification multi-étiquettes), la recherche et le classement. Dans ce cadre, nous limiterons le travail à des problèmes de régression et de classification.

Afin de rendre le travail d'implémentation accessible et efficace, nous utiliserons une bibliothèque Python (scikit-learn) intégrant les méthodes d'apprentissage, des objets de type perceptron et perceptron multi-couches, ainsi que des méthodes d'évaluation. Pour commencer, il est nécessaire de procéder à l'installation de la bibliothèque.

```
pip install sklearn
```

Le perceptron

Dans un premier temps, nous nous pencherons sur l'entraînement d'un perceptron pour l'implémentation des opérateurs logiques OR, AND et XOR. Ces opérateurs ont été paramétrés manuellement au début du cours. Vous pourrez ainsi comparer les paramètres obtenus par apprentissage automatique à ceux définis manuellement.

Pour entamer l'apprentissage automatique, suivez la procédure détaillée ci-dessous :

- Dans un script python, commencez par importer la définition du perceptron de la bibliothèque scikit-learn

```
from sklearn.linear_model import Perceptron
```

- Définissez toutes les valeurs possibles pour les entrées de l'opérateur logique

```
#liste d'entrées possibles pour l'opérateur logique  
data = [[0,0], [0,1], [1,0], [1,1]]
```

- Définissez les labels en sortie attendus selon l'opérateur concerné (ex. pour l'opérateur OR) :

```
#labels correspondants selon liste d'entrées  
ORLabels = [0, 1, 1, 1]
```

- Créez une instance du perceptron et indiquez le nombre d'itérations max. admis pour l'apprentissage ainsi que le seuil de d'erreur toléré (paramètres de convergence).

```
#Création du perceptron  
operateurOR = Perceptron(max_iter=40, tol=1e-3)
```

- Exécutez la procédure d'apprentissage en utilisant la fonction **fit** de la variable perceptron et en indiquant en paramètres les entrées et les labels préalablement définis.
- La fonction **predict** renvoie la valeur en sortie du réseau pour une entrée spécifiée.

Exercices de classification

1. Concevez et entraînez les perceptrons pour les opérateurs logiques OU, ET et OU-Exclusif.
2. Vérifiez les prédictions du perceptron à l'aide de la fonction **predict**. Est-ce que toutes les prédictions sont correctes? Oui, non, pourquoi?
3. Utilisez la fonction score pour estimer la précision moyenne du perceptron après l'entraînement. Rapportez les résultats. Le score reflète-t-il la qualité des résultats du perceptron? Quelle est la plage de valeurs du score rapporté par la bibliothèque?

Exercice d'OU-Exclusif comme un classifieur

4. Comme constaté dans l'exercice précédent, l'opérateur OU-Exclusif ne peut pas être satisfait par un classifieur linéaire. Il est donc nécessaire d'utiliser une topologie de perceptron multi-couche. Pour ce faire, utilisez le MLPClassifier de la bibliothèque scikit-learn.

```
#Importation de la définition du perceptron multi-couche
from sklearn.neural_network import MLPClassifier
```

5. Choisissez une fonction d'activation type **tangente hyperbolique** ('tanh'), **une couche** cachées et deux neurones et une **limite d'itérations** de 10000. Référez-vous à la documentation : [lien](#).

Est-ce que le modèle converge lors de l'étape d'apprentissage? Comparez les valeurs prédites par le réseau et celles attendues. Est-ce que son score atteint la valeur de 1? Est-ce que les résultats de l'entraînement du réseau sont les mêmes à chaque cycle d'apprentissage? Faites plusieurs essais.

6. A partir des résultats obtenus dans l'exo. 5, indiquez les poids obtenus dans le modèle topologique de la Fig. 1. Les poids du réseau correspondent à l'attribut **coefs_** de l'objet ($w_{1a}, w_{1b}, w_{2a}, w_{2b}, w_3, w_4$). Les biais du réseau sont définis dans l'attribut **intercepts_** (b_1, b_2, b_3).

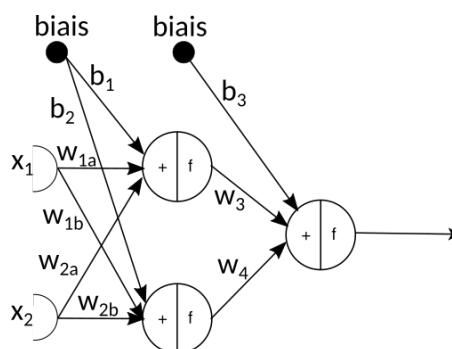


Fig. 1 : Topologie du perceptron multi-couche - Opérateur logique OU-Exclusif

Exercice d'OU-Exclusif comme une régression

Une approche alternative dans l'implémentation de l'opérateur logique OU-Exclusif consiste à le représenter sous la forme d'une régression. Dans ce cas, le réseau devra prédire les valeurs de sortie souhaitées plutôt que la classe, contrairement au cas du classifieur.

7. - L'entité disponible à cet objet dans la bibliothèque scikit-learn est celle du MLPRegressor :

```
#Importation de la définition du perceptron multi-couche pour la régression
from sklearn.neural_network import MLPRegressor
```

- Définissez une instance en indiquant un minimum de 2 couches cachées, une fonction d'activation 'tanh' et l'utilisation de la méthode d'optimisation 'lbfgs'.
- Entraînez le réseau (fit) en indiquant les entrées et la valeur de sortie correspondante.
- Estimez le score et vérifiez les prédictions.

Exercice d'application robotique

8. Implantez un perceptron multicouche de régression permettant le suivi d'une ligne en utilisant en entrée les mesures normalisées des capteurs prévus à cet effet dans le robot Thymio (voir variable "prox.ground.delta"). Le réseau devra prédire les vitesses de moteurs appropriées (normalisées dans une gamme $[-1,1]$).
Rapportez la topologie (graphe), les poids, les biais et la fonction d'activation.
Validez votre proposition à l'aide de la précision moyenne (score) en indiquant les vecteurs d'entraînement choisis. Contrôlez sa validité à l'expérimental et rapportez une vidéo illustrant le fonctionnement.
 - (a) Programmez un script pour la collecte de données capteurs/actionneurs en utilisant une stratégie de suivi de ligne simple.
 - (b) Utilisez la base de données collectée pour effectuer l'apprentissage supervisé d'un MLP.
 - (c) Validez à l'aide du score la performance du réseau obtenu.
 - (d) Utilisez le MLP pour contrôler le robot et validez le suivi de ligne.
9. Implantez dans un seul réseau multicouche, une stratégie de suivi de ligne avec évitement d'obstacles. Le robot devra ainsi être capable d'éviter des obstacles placés sur la ligne. Pour ce faire, le réseau pourra faire usages de proximètres et des capteurs au sol.